ЛАБОРАТОРНАЯ РАБОТА № 4

Ответы к защите

Основы MLOps

Вариант 17
Прогноз выработки электростанции
Combined Cycle Power Plant

29 вопросов по отчёту	20 вопросов по теории

Команды Ubuntu, объяснения кода, фактические метрики, MLflow Model Registry, API, Docker, Compose и откат.

Alexanderio1/AIMLLab4

Подготовлено 11 ноября 2026 года

Содержание

Вводные сведения и результаты проекта **5**

Как пользоваться ответами 5

Числа и идентификаторы, которые нужно знать 6

I. Вопросы по отчёту для защиты лабораторной работы **7**

П1. Какой командой перейти в корневую папку проекта? Как проверить текущий путь? 7

П2. Как создать виртуальное окружение Python и активировать его в Ubuntu? 7

П3. Как выбрать интерпретатор виртуального окружения в VS Code? 8

П4. Как установить зависимости из requirements.txt? Как проверить установленные версии пакетов? 8

П5. Какой командой проверить работоспособность Docker? Как посмотреть версию Docker Compose? 9

П6. Как запустить сервер MLflow? 9

П7. По какому адресу открывается интерфейс MLflow? Как проверить доступность сервера из терминала? 9

П8. Как задать MLFLOW_TRACKING_URI? В каком терминале нужно выполнить эту команду? 10

П9. Как в вашем коде загружаются данные и выделяются обучающая, валидационная и тестовая выборки? 10

П10. Где задаётся random_state? Как в отчёте подтвердить состав разбиения? 11

П11. Как запустить обучение с заданными гиперпараметрами через аргументы командной строки? Приведите пример команды. 11

П12. Какие настройки вы изменяли в шести экспериментах? Какие условия сравнения сохраняли одинаковыми? 12

П13. Какие вызовы MLflow записывают параметры, метрики, теги и файлы? Покажите соответствующие строки кода. 13

П14. Где в интерфейсе MLflow найти результаты запуска, его run_id и сохранённые артефакты? . . 14

П15. Как сравнить несколько запусков в MLflow? По какому показателю вы выбрали модель? 14

П16. Как зарегистрировать модель в Model Registry? Как узнать фактический номер созданной версии? .

П17. Как назначить alias champion выбранной версии? Как проверить результат назначения? 15

П18. В каком порядке выполняются экспорт модели и её итоговая оценка? Какие команды для этого используются? 16

П19. Какие файлы появляются после экспорта? Для чего нужны metadata.json, sample_request.json и expected_predictions.json? . . . 16

П20. Какие инструкции содержит ваш Dockerfile? Что выполняют FROM, WORKDIR, COPY, RUN, USER, EXPOSE и CMD? . . . 17

П21. Как собрать Docker-образ с заданным тегом? Что означает точка в конце команды сборки? . . 17

П22. Как проверить загрузку экспортированной модели внутри образа до запуска API? . . . 18

П23. Как запустить контейнер в фоновом режиме с публикацией порта только на локальном адресе компьютера? . . . 18

П24. Как посмотреть работающие и остановленные контейнеры? Как получить логи конкретного контейнера? . . . 19

П25. Как проверить /health и /metadata через curl? Какие сведения должны содержать ответы? . . . 19

П26. Как отправить POST-запрос с JSON из файла через curl? Как выполнить аналогичный запрос через /docs? . . . 20

П27. Как проверить обработку некорректных запросов и совпадение предсказаний API с локальной моделью? . . . 20

П28. В каком порядке выполняются обновление модели, экспорт, сборка нового образа и замена контейнера? Как затем выполнить откат? . . . 21

П29. Как запустить сервис через Docker Compose и проверить статус healthcheck? Как посмотреть его логи после удаления ручного контейнера? . . . 21

II. Вопросы по теории для защиты лабораторной работы 23

Т1. Что такое MLOps? Какие этапы жизненного цикла модели он охватывает? . . . 23

Т2. Чем разработка модели машинного обучения отличается от её эксплуатации? . . . 23

Т3. Что означает воспроизводимость эксперимента? Какие сведения необходимы для повторения результата? . . . 24

Т4. Что такое MLflow? Для чего предназначены Tracking, Models и Model Registry? . . . 24

Т5. Чем experiment, run, сохранённая модель и Registry-версия отличаются друг от друга? . . . 25

Т6. Чем параметры, гиперпараметры, метрики, теги и артефакты отличаются друг от друга? . . . 26

Т7. Что такое происхождение модели — model lineage? Зачем связывать модель с кодом, данными и экспериментом? . . . 26

Т8. Для чего нужны версии моделей и aliases? Чем изменяемая ссылка отличается от конкретного номера версии? . . . 27

Т9. Зачем разделять данные на train, validation и test? Какую роль выполняет каждая выборка? . . . 27

Т10. Что такое утечка данных? Как она возникает при подготовке признаков и выборе модели? . . 28

Т11. Что такое Pipeline? Почему преобразования данных необходимо сохранять вместе с моделью? . 28

Т12. Что такое сериализация модели? Какие проблемы совместимости и доверия возникают при её загрузке? 29

Т13. Что такое Docker? Чем образ, контейнер и Dockerfile отличаются друг от друга? 29

Т14. Чем контейнеризация отличается от виртуальной машины и виртуального окружения Python? . 30

Т15. Что такое контекст сборки, слои образа и кеш Docker? Для чего нужен .dockerignore? 30

Т16. Чем тег образа отличается от Image ID и digest? Как это связано с воспроизводимостью развёртывания? 31

Т17. Как устроена публикация портов контейнера? Чем адреса 127.0.0.1 и 0.0.0.0 отличаются при настройке сервиса? 31

Т18. Что такое HTTP API модели? Для чего нужны валидация входных данных и описание формата запросов и ответов? 32

Т19. Что проверяют healthcheck, readiness и liveness? Почему работоспособность сервиса не подтверждает качество модели? 32

Т20. Что такое мониторинг модели после развёртывания? Зачем отслеживать ошибки, задержки, качество предсказаний и дрейф данных? 33

III. Краткий план ответа о собственной работе **34**

IV. Источники и навигация по проекту **35**

Вводные сведения и результаты проекта

Основы MLOps. Вариант 17: прогноз выработки электростанции CCPP.

29 вопросов по отчёту и 20 вопросов по теории. Подготовлено 11 ноября 2026 года. Ответы относятся к опубликованному проекту Alexanderio1/AIMLLab4 и результатам его выполнения на Ubuntu 22.04.5 ARM64. Команды ниже — примеры для демонстрации и объяснения; подготовка этого файла не запускала новое обучение или Docker.

Как пользоваться ответами

Сначала изучите краткую формулировку ответа, затем команду и её параметры. На защите связывайте теорию с собственными файлами и измерениями. Ссылки на строки кода приведены для состояния репозитория `fec4e2a`; при последующих изменениях надёжнее ориентироваться также на названия функций.

Все команды рассчитаны на корень рабочей копии проекта. Исходная папка Ubuntu: `~/lab4-ubuntu22/AIMLLab4`. Для нового опыта нельзя автоматически использовать старые `reports/registry.json` как действующий Registry: это только сохранённые сведения. База SQLite и бинарные модели должны присутствовать в исходной Ubuntu или быть восстановлены из её остановленной резервной копии. Для нового обучения из опубликованного репозитория предусмотрен отдельный чистый каталог:

```
bash scripts/create_run.sh "$HOME/AIMLLab4-new-run"
cd "$HOME/AIMLLab4-new-run"
bash scripts/install.sh
```

В PDF код может переноситься по ширине страницы. Для копирования длинных команд доступен исходный Markdown `answers_lab4.md` в том же репозитории.

Числа и идентификаторы, которые нужно знать

Объект	Фактическое значение
Источник	UCI Combined Cycle Power Plant, ID 294
Признаки → цель	AT, V, AP, RH → PE
Единицы	AT: °C; V: cm Hg; AP: mbar; RH: %; PE: MW
Таблица	Только Sheet1 файла Folds5x2_pp.xlsx
Строки	9568 исходных; 41 полный дубликат; 9527 после очистки
Разбиение	5715 train / 1906 validation / 1906 test; seed=42
Сравнение	3 Ridge и 3 RandomForestRegressor; минимум validation MAE
Победитель	RF, 250 деревьев, max_depth=None, min_samples_leaf=1
Validation	MAE=2,472198 МВт; RMSE=3,505922 МВт; R^2=0,957505
Итоговый test	MAE=2,490805 МВт; RMSE=3,601666 МВт; R^2=0,956215
Registry	ccpp-power; version 1 — Ridge; version 2 — RF
Alias после Compose	champion → version 2
Docker	ccpp-api:v1 → ccpp-api:v2 → откат ccpp-api:v1
Платформа	Ubuntu 22.04.5 ARM64; проект Python 3.12.14
Инструменты	MLflow 3.5.0; sklearn 1.7.2; Docker 29.9.3; Compose v5.7.0

Run ID выбранного RF: `503b388935b047bca0fa775e9603e454`. Run ID базового Ridge: `22d806e000234fa38084837219197507`. Влажность в исходнике достигает 100,16%: 55 исходных строк выше 100% сохранены, а не обрезаны. PE измеряется в МВт, это мощность; МВт·ч — единица энергии.

Метрики: $MAE = (1/n)\ \Sigma_i\ |y_i - \hat{y}_i|$; $RMSE = \sqrt{[(1/n)\ \Sigma_i\ (y_i - \hat{y}_i)^2]}$; $R^2 = 1 - [\Sigma_i\ (y_i - \hat{y}_i)^2] / [\Sigma_i\ (y_i - \bar{y})^2]$. Здесь n — число наблюдений, y_i — фактическая мощность, $\hat{y}_i$ — прогноз, $\bar{y}$ — среднее фактических значений. MAE/RMSE имеют единицы цели и предпочтительны меньшие; R^2 безразмерен и предпочтителен больший. R^2 может быть отрицательным, если прогноз хуже постоянного среднего на оцениваемой выборке.

I. Вопросы по отчёту для защиты лабораторной работы

П1. Какой командой перейти в корневую папку проекта? Как проверить текущий путь?

Короткий ответ: перейти командой `cd`, проверить путь командой `pwd`, а наличие файлов проекта — командой `ls`.

```
cd ~/lab4-ubuntu22/AIMLLab4
pwd
ls
```

Символ `~` означает домашнюю папку текущего пользователя. В исходной Ubuntu `pwd` показывал `/home/alexanderio/lab4-ubuntu22/AIMLLab4`. Корень проекта определяется не словом «корень» в имени, а наличием `src/`, `scripts/`, `tests/`, `Dockerfile`, `compose.yaml`, `requirements.txt`. Это не системный каталог `/`.

Относительные пути, например `data/split.json`, отсчитываются от текущей папки. Если находиться внутри `src`, команда `bash scripts/run.sh experiments` будет искать несуществующую вложенную папку. Поэтому перед демонстрацией сначала проверяю `pwd`. Скрипты проекта дополнительно определяют свой корень по расположению файла, но при ручных командах это не отменяет проверки пути.

Что показать: скриншот `01-system.png`, структуру репозитория и вывод `pwd`. Если используется новый каталог, созданный `create_run.sh`, команда `cd` должна указывать именно на него, а не на исходную папку со старыми результатами.

П2. Как создать виртуальное окружение Python и активировать его в Ubuntu?

При доступном Python 3.12 стандартный вариант:

```
python3.12 -m venv .venv
source .venv/bin/activate
python --version
which python
```

`venv` создаёт отдельный каталог интерпретатора/пакетов, а `source` выполняет скрипт активации в текущем shell и добавляет `.venv/bin` в начало PATH. Активация действует только в этом терминале. Ожидается Python 3.12 и путь, оканчивающийся на `.venv/bin/python`. Для выхода выполняют `deactivate`.

Особенность моей Ubuntu: в Ubuntu 22.04 системный Python — 3.10.12, поэтому команда `python3 -m venv .venv` сама по себе создала бы окружение 3.10. В проекте выполнялась `bash scripts/install.sh`: при отсутствии Python 3.12 она вызывает `bootstrap_python.sh`, устанавливает uv в `.tools/uv`, отдельный CPython 3.12.14 в `.tools/python` и создаёт `.venv`. Системный Python не заменяется.

```
bash scripts/install.sh
python3 --version
.venv/bin/python --version
```

В нашем случае эти две проверки показывают соответственно 3.10.12 и 3.12.14. Прямой вызов `.venv/bin/python` работает и без активации; именно так написаны основные скрипты.

Не нужно устанавливать пакеты через `sudo pip` или менять `/usr/bin/python3`: это может нарушить работу системных инструментов Ubuntu.

П3. Как выбрать интерпретатор виртуального окружения в VS Code?

Открываю в VS Code корневую папку проекта, устанавливаю расширение Python от Microsoft и вызываю палитру команд `Ctrl+Shift+P`. Выбираю **Python: Select Interpreter**, затем интерпретатор из `.venv`. Если его нет в списке, использую **Enter interpreter path** и указываю полный путь:

```
/home/alexanderio/lab4-ubuntu22/AIMLLab4/.venv/bin/python
```

После выбора проверяю строку состояния редактора и открываю новый встроенный терминал. Старый терминал может сохранять прежний PATH. Проверка:

```
python -c "import sys; print(sys.executable); print(sys.version)"
```

Путь должен вести в выбранную `.venv`, версия — 3.12.14. Выбор интерпретатора нужен для запуска Python, подсказок, отладки и поиска импортов. Само наличие виртуального окружения в папке не гарантирует, что редактор использует его.

Если открыт Jupyter notebook, отдельно проверяю **Select Kernel** в правой верхней части notebook: kernel и интерпретатор обычного Python-файла могут отличаться. Для kernel в окружении должен быть установлен ipykernel.

По фактической работе: вход в Codex внутри VS Code не использовался. Обучение и Docker выполнены через терминал Ubuntu; интерфейсы MLflow/FastAPI — через Firefox. Работа VS Code не является обязательным условием работы проекта.

П4. Как установить зависимости из requirements.txt? Как проверить установленные версии пакетов?

Надёжно вызывать pip через тот Python, для которого устанавливаются пакеты:

```
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python -m pip check
.venv/bin/python -m pip list
.venv/bin/python -m pip show mlflow scikit-learn
```

`-r` означает чтение списка зависимостей из файла. `pip check` проверяет совместимость объявленных зависимостей, но не заменяет тестирование приложения. `pip list` показывает версии, `pip show` — также расположение пакета и метаданные. Дополнительно можно проверить импорты:

```
.venv/bin/python -c 'import mlflow, sklearn; print(mlflow.__version__, sklearn.__version__)'
```

В нашей работе `requirements.txt` фиксирует прямые зависимости, а `requirements.lock.txt` — полный снимок, включая транзитивные пакеты. Установщик использует именно lock-файл:

```
.venv/bin/python -m pip install -r requirements.lock.txt
.venv/bin/python -m pip freeze > reports/installed-requirements.txt
```

Выполнение `freeze` здесь показано для рабочей папки нового опыта; в опубликованной папке отчёта сохранённый снимок не перезаписываю. Фактические ключевые версии: MLflow 3.5.0, sklearn 1.7.2, pandas 2.3.3, numpy 2.2.6, FastAPI 0.119.0. Контейнер использует отдельный `requirements-runtime.lock.txt`; версии библиотек предсказания согласованы с обучением. Зависимости генерации отчёта вынесены в `requirements-report.txt` и не являются новыми гиперпараметрами модели.

П5. Какой командой проверить работоспособность Docker? Как посмотреть версию Docker Compose?

Для проверки клиента, демона и запуска контейнера:

```
sudo docker version
sudo docker info
sudo docker run --rm hello-world
sudo docker compose version
```

`docker version` должен показать не только Client, но и Server. Один вывод `docker --version` подтверждает наличие программы-клиента, но не доступность демона. `docker info` проверяет соединение с Docker Engine. `hello-world` проверяет получение/наличие образа и фактический запуск контейнера; при первой загрузке дополнительно нужен доступ к реестру. `--rm` удаляет завершившийся тестовый контейнер, но не весь Docker-кеш.

Если демон не запущен в Ubuntu с systemd, проверяю `sudo systemctl status docker`. Ошибка permission denied относится к доступу к сокету Docker: в этой работе использовался sudo, добавление в группу docker не было обязательным.

В исходном выполнении Docker Engine — **29.9.3**, Docker Compose — **v5.7.0**, архитектура linux/arm64. Команда современного плагина пишется `docker compose` с пробелом; это не старый отдельный Python-инструмент `docker-compose`.

П6. Как запустить сервер MLflow?

Основной способ проекта:

```
.venv/bin/python scripts/mlflow_server.py start
```

Скрипт запускает MLflow в фоне, сохраняет PID в `mlflow-data/server.pid`, пишет лог в `mlflow-data/server.log` и ожидает HTTP 200 от `/health`. Метаданные хранятся в SQLite, модели/файлы — в каталоге artifacts. Команда `bash scripts/run.sh experiments` сама вызывает этот запуск до обучения.

Эквивалентный ручной запуск в отдельном терминале:

```
mkdir -p mlflow-data/artifacts
.venv/bin/mlflow server \
  --host 127.0.0.1 --port 5000 --workers 1 \
  --backend-store-uri "sqlite:///$PWD/mlflow-data/mlflow.db" \
  --serve-artifacts \
  --artifacts-destination "$PWD/mlflow-data/artifacts"
```

`--backend-store-uri` задаёт базу метаданных; `--serve-artifacts` включает проксирование артефактов сервером, `--artifacts-destination` — их локальное хранилище. Абсолютный путь после `sqlite:///` даёт четыре слеша перед `home`.

Нужно выбрать один способ. Если первый сервер уже занял порт 5000, второй не запускаю параллельно. Ручной foreground-процесс останавливается Ctrl+C; сервер, созданный скриптом, — через `mlflow_server.py stop`. В исходной SQLite должны присутствовать run и версии: запуск пустого сервера после клонирования Git не восстановит их из `registry.json`.

П7. По какому адресу открывается интерфейс MLflow? Как проверить доступность сервера из терминала?

Интерфейс открывается в браузере Ubuntu:

```
http://127.0.0.1:5000
```

Проверка из терминала:

```
curl -i --max-time 5 http://127.0.0.1:5000/health
curl -fsS --max-time 5 -o /dev/null \
  -w 'HTTP %{http_code}\n' http://127.0.0.1:5000/health
```

Ожидается HTTP 200. Первый вариант показывает заголовки и тело; второй удобен для автоматической проверки и возвращает ошибку при HTTP-ошибке или проблеме соединения. В скрипте та же идея реализована через `requests.get(..., timeout=2)`.

Connection refused означает, что по указанному адресу никто не принимает соединение; проверяю запуск MLflow и `mlflow-data/server.log`. Если порт занят чужим процессом, не завершаю его вслепую. При необходимости исследую `ss -ltnp` и согласованно выбираю другой порт для сервера и клиента.

Адрес 127.0.0.1 относится к тому компьютеру, где открыт браузер. На Mac он указывает на macOS, а не автоматически на Ubuntu-виртуалку. В этой лабораторной браузер был запущен внутри Ubuntu. `/health` проверяет готовность сервера, но не доказывает наличие конкретной зарегистрированной версии модели.

П8. Как задать MLFLOW_TRACKING_URI? В каком терминале нужно выполнить эту команду?

```
export MLFLOW_TRACKING_URI=http://127.0.0.1:5000
```

Команду выполняю в терминале, из которого будут запущены обучение, регистрация, оценка или экспорт. Дочерние процессы наследуют экспортированную переменную. Другой уже открытый терминал её не получит: в нём export нужно повторить. Если сервер запущен в терминале A, а train — в терминале B, важен export в B.

Проверить значение можно так:

```
printf '%s\n' "$MLFLOW_TRACKING_URI"
.venv/bin/python -c 'import os; print(os.getenv("MLFLOW_TRACKING_URI"))'
```

В `connect()` используется `mlflow.set_tracking_uri` со значением этой переменной; при её отсутствии задан тот же default `http://127.0.0.1:5000`. В `scripts/run.sh` export выполняется автоматически перед любым этапом.

Tracking URI — адрес API клиента, а не путь хранения SQLite. Для нашего сервера клиенту нужен HTTP URI; путь SQLite передаётся самому серверу. Во время предсказаний Docker API не обращается к MLflow, поскольку содержит локальный экспорт модели. Поэтому этому контейнеру Tracking URI для инференса не нужен.

П9. Как в вашем коде загружаются данные и выделяются обучающая, валидационная и тестовая выборки?

В `prepare()` скачивается официальный ZIP UCI, из него извлекается `Folds5x2_pp.xlsx`. Читается **первый лист**, то есть Sheet1:

```
excel = pd.ExcelFile(workbook, engine="openpyxl")
selected_sheet = excel.sheet_names[0]
original = pd.read_excel(excel, sheet_name=selected_sheet)
clean = original.drop_duplicates().copy()
clean.insert(0, "row_id", clean.index.astype(int))
clean = clean.reset_index(drop=True)
```

До очистки проверяются схема, 9568 строк, пропуски и конечность чисел. После удаления 41 полного дубликата остаётся 9527 наблюдений. RH до 100,16% сохраняется; введённая сначала теоретическая граница 100% была исправлена до успешного обучения. Пять листов — перестановки одной таблицы, их не объединяли.

В `split_frame()` одинаковые векторы четырёх признаков хешируются в группы. Первый `GroupShuffleSplit(test_size=0.2)` отделяет test, второй `GroupShuffleSplit(test_size=0.25)` делит оставшуюся development-часть на train и validation. Поэтому получаются примерно 60/20/20, фактически **5715/1906/1906**. Группировка не допускает одинаковые X в разных частях, даже если значения PE у таких строк различались бы.

`dataset(part)` читает соответствующий CSV и возвращает только `data[FEATURES]` и `data[TARGET]`. `row_id` служебный и в X не входит. Для Ridge стандартизация обучается внутри Pipeline на train. Test не читается в `train()`, он нужен только в `evaluate()` после выбора модели.

П10. Где задаётся random_state? Как в отчёте подтвердить состав разбиения?

В `src/workflow.py` объявлено `SEED = 42`. Это значение передаётся обоим `GroupShuffleSplit` и `RandomForestRegressor`. У Ridge в нашей конфигурации используется детерминированный решатель по умолчанию; отдельный random_state для него не задавался. Seed логируется в параметры MLflow и аудит.

```
first = GroupShuffleSplit(
    n_splits=1, test_size=.2, random_state=SEED
)
second = GroupShuffleSplit(
    n_splits=1, test_size=.25, random_state=SEED
)
```

Одинакового seed недостаточно для восстановления split, если изменились данные, порядок строк или алгоритм разбиения. Поэтому сохранены:

- `data/split.json` — исходные row_id каждой части, seed и метод;
- `data/processed/train.csv`, `validation.csv`, `test.csv` — сами части;
- `reports/data_audit.json` — размеры и SHA-256 исходника, данных и split;
- артефакт `split.json` каждого MLflow run.

```
cat data/split.json
cat reports/data_audit.json
.venv/bin/python scripts/validate_artifacts.py
```

Последняя команда проверяет опубликованные результаты без обучения: заново восстанавливает seed=42, сверяет индексы и отсутствие пересечений. Скриншот `05-data.png` показывает размеры, но точный состав подтверждает JSON/CSV, а не только надпись «60/20/20». Случайный split без временных меток не доказывает качество прогноза будущих периодов эксплуатации станции.

П11. Как запустить обучение с заданными гиперпараметрами через аргументы командной строки? Приведите пример команды.

Важное отличие нашего проекта: базовая команда запускает все шесть заранее описанных конфигураций. Парсер `workflow.py` имеет подкоманду `train`, но не имеет флагов `--alpha`, `--trees` или `--algorithm` для неё:

```
.venv/bin/python src/workflow.py --help
.venv/bin/python src/workflow.py train --help
.venv/bin/python src/workflow.py train
```

Гиперпараметры исходных шести run заданы в функции `configurations()`. Команду вроде `workflow.py train --alpha 1` для опубликованного парсера приводить как рабочую было бы неверно: он вернёт ошибку неизвестного аргумента. Повторный полный train выполняется в чистом рабочем каталоге, чтобы не перезаписать опубликованные таблицы.

Для демонстрации принципа передачи аргументов можно выполнить следующий самодостаточный пример. Он использует функции нашего проекта, принимает `--algorithm` и `--alpha`, обучается только на train, печатает validation-метрики и не меняет сохранённую таблицу шести запусков или Registry:

```
.venv/bin/python - --algorithm ridge --alpha 1.0 <<'PY'
import argparse
import sys
sys.path.insert(0, "src")
import workflow as w

p = argparse.ArgumentParser()
p.add_argument("--algorithm", choices=["ridge", "rf"], required=True)
p.add_argument("--alpha", type=float, default=1.0)
p.add_argument("--trees", type=int, default=200)
p.add_argument("--depth", type=int, default=16)
a = p.parse_args()
algorithm = "Ridge" if a.algorithm == "ridge" else "RandomForestRegressor"
params = ({"alpha": a.alpha} if a.algorithm == "ridge" else
          {"n_estimators": a.trees, "max_depth": a.depth})
X, y = w.dataset("train")
Xv, yv = w.dataset("validation")
model = w.make_model(algorithm, params)
model.fit(X, y)
print(w.metrics(yv, model.predict(Xv), "val"))
PY
```

Здесь `python -` читает программу из stdin, а аргументы после `-` разбираются `argparse`. Для RF заменяют первую строку на `--algorithm rf --trees 200 --depth 16`. Это дополнительный учебный пример, не команда исторических шести MLflow run; в нём нет логирования MLflow. Для постоянного CLI такой код можно вынести в отдельный файл и дополнить теми же log-вызовами, что в `train()`.

П12. Какие настройки вы изменяли в шести экспериментах? Какие условия сравнения сохраняли одинаковыми?

Запуск	Алгоритм	Изменяемые гиперпараметры	Validation MAE, МВт
ridge-a0.1	Ridge	alpha=0.1	3,644305
ridge-a1	Ridge	alpha=1.0	3,644376
ridge-a10	Ridge	alpha=10.0	3,645504
rf-depth8	RF	120 деревьев; depth=8; leaf=2	2,822423
rf-depth16	RF	200 деревьев; depth=16; leaf=1	2,481119
rf-unlimited	RF	250 деревьев; depth=None; leaf=1	2,472198

Для Ridge alpha определяет силу L2-регуляризации; StandardScaler одинаков во всех трёх Ridge. У RF меняются `n_estimators`, `max_depth`, `min_samples_leaf`. При ограничении глубины 8 модель менее гибкая.

Неизменными остаются исходные данные и очистка, признаки/цель и единицы, состав train/validation/test, seed, основная метрика MAE на validation, версии библиотек и протокол логирования. Для RF сохраняются seed=42 и n_jobs=2. Различие preprocessing между семействами обосновано: Ridge чувствителен к масштабу при регуляризации, деревьям StandardScaler для выбора порогов не нужен.

Победитель по validation MAE — RF без ограничения глубины. Его преимущество над лучшим Ridge около 32,16%; отрыв от RF depth16 всего 0,008920 МВт. Статистическая значимость этого малого отрыва не проверялась. `fit_seconds` измеряет только fit; длительность run в UI включает также сериализацию и логирование артефактов и потому может быть существенно больше.

П13. Какие вызовы MLflow записывают параметры, метрики, теги и файлы? Покажите соответствующие строки кода.

Основное логирование находится в `src/workflow.py`, функция `train()`, строки 169–209. Сначала задаётся experiment и открывается run:

```
mlflow.set_experiment(EXPERIMENT)
with mlflow.start_run(run_name=name) as run:
    ...  # fit, логирование и сохранение модели; тело сокращено
```

Параметры, метрики и теги записывают разные методы. Ниже те же вызовы, с сокращёнными словарями и переносами для читаемости:

```
mlflow.log_params({
    "algorithm": algorithm, **params, "seed": SEED,
    "n_train": len(X), "n_validation": len(Xval)
})
mlflow.log_metrics({**values, "fit_seconds": elapsed})
mlflow.set_tags({
    "variant": "17", "selection_metric": "val_mae",
    "target": "PE", "target_unit": "MW"
})
```

`values` содержит val_mae, val_rmse, val_r2. В полном коде дополнительно логируются размеры test, preprocessing, n_jobs, хеши данных/split и единицы. Файлы и структурированные артефакты:

```
mlflow.log_dict(UNITS, "units.json")
mlflow.log_dict(audit, "data_audit.json")
mlflow.log_artifact("data/split.json")
mlflow.log_artifact("requirements.lock.txt", artifact_path="source")
mlflow.log_artifact(__file__, artifact_path="source")
mlflow.log_figure(fig, "validation_predictions.png")
```

Pipeline сохраняется как MLflow Model со схемой входа и примером:

```
info = mlflow.sklearn.log_model(
    model, name="model", input_example=X.head(3),
    signature=infer_signature(X, model.predict(X)),
    pip_requirements=Path("requirements.lock.txt").read_text().splitlines()
)
```

`mlflow.log_input(..., context="training")` связывает run с dataset input. Run ID берётся из `run.info.run_id`, URI модели — из `info.model_uri`. После выбора `evaluate()` открывает тот же run победителя по `run_id` и дописывает test-метрики; это не обучение новой модели.

П14. Где в интерфейсе MLflow найти результаты запуска, его run_id и сохранённые артефакты?

Открываю `http://127.0.0.1:5000`, выбираю experiment `lab4-ccpp-variant17` и вкладку Runs. В таблице каждая строка — один запуск; щелчок по имени, например `rf-unlimited`, открывает подробности.

В карточке run находятся его ID, статус, параметры, метрики, теги и ссылки на связанные модели. Артефакты run доступны в разделе Artifacts: там ищу `split.json`, `units.json`, `data_audit.json`, исходники и график. В MLflow 3 логированная модель — отдельная сущность: если её файлы не видны как привычная папка `artifacts/model`, перехожу по ссылке model/вкладке Models для связанной logged model. Не следует придумывать путь `runs:/.../model`, если фактически записан URI `models:/m-...`.

В нашем победителе run_id равен `503b388935b047bca0fa775e9603e454`. Его можно также увидеть в `reports/experiments.csv`, `reports/registry.json` и метаданных экспорта v2. Через Python, при доступном исходном MLflow:

```
.venv/bin/python - <<'PY'
import mlflow
mlflow.set_tracking_uri("http://127.0.0.1:5000")
r = mlflow.get_run("503b388935b047bca0fa775e9603e454")
print(r.info.run_id)
print(r.data.params)
print(r.data.metrics)
PY
```

Скриншот `07-mlflow-runs.png` показывает шесть успешных run. Число запусков и сохранённый CSV не означают, что после остановки/переноса машины сервер всё ещё запущен: для просмотра UI его запускают заново с исходной базой.

П15. Как сравнить несколько запусков в MLflow? По какому показателю вы выбрали модель?

В таблице Runs включаю через Columns `val_mae`, `val_rmse`, `val_r2` и нужные params. Выделяю несколько run флажками и открываю Compare, если нужно сравнение параметров и графиков. Для общего ранжирования сортирую `val_mae` по возрастанию. Нужно сравнивать одинаковые данные и выборки, а не смешивать test одной модели с validation другой.

Критерий зафиксирован заранее: **минимум validation MAE в МВт**. MAE даёт среднюю абсолютную ошибку мощности в понятных предметных единицах. RMSE и R^2 дополнительно описывают качество, но не меняют правило выбора. При точном равенстве MAE код дополнительно сортирует по имени run — это детерминированный способ разрешить ничью, а не новый научный критерий.

Результат: rf-unlimited с MAE=2,472198 МВт; лучший Ridge — 3,644305 МВт. Test оценивался только у уже выбранной модели: MAE=2,490805 МВт, RMSE=3,601666 МВт, R^2=0,956215. Если после просмотра test начать менять гиперпараметры и снова выбирать по нему, test перестанет быть независимым.

Разница с RF depth16 мала и сама по себе не доказывает статистического превосходства. Более простая модель могла бы стать выбором при заранее согласованном ограничении размера/задержки, но в этой работе такого вторичного правила не было и минимальная MAE определила победителя.

П16. Как зарегистрировать модель в Model Registry? Как узнать фактический номер созданной версии?

В проекте для этого предусмотрена команда:

```
.venv/bin/python src/workflow.py register
```

Функция `register()` читает отсортированные результаты. Сначала регистрирует лучший запуск другого алгоритма как baseline, затем победителя как selected. Основной вызов и способ получить номер:

```
mv = mlflow.register_model(
    record["model_uri"], "ccpp-power",
    await_registration_for=120
)
version = str(mv.version)
```

Источник — URI реально сохранённой MLflow-модели. В нашем RF это `models:/m-7a02a502abe74b62b1bd4704453e6475`. Возвращаемый объект ModelVersion содержит фактический номер. Код сохраняет `mv.version` в `reports/registry.json`, а также печатает его. Номера видны в разделе Models → ccpp-power и на скриншоте `08-registry.png`.

В исходном опыте созданы version 1 (Ridge) и version 2 (RF). Нельзя всегда присваивать номер «2» по предположению: в существующем Registry следующая версия может иметь другой номер. Повторный `register` создаёт новые версии, поэтому для простого просмотра его не запускают заново. JSON — историческая запись; актуальное состояние узнают у сервера, например через `MlflowClient().search_model_versions("name='ccpp-power'")`.

П17. Как назначить alias champion выбранной версии? Как проверить результат назначения?

При доступном MLflow можно назначить alias через клиент. Пример берёт номер из сохранённого результата выбранной версии, а не жёстко предполагает его:

```
.venv/bin/python - <<'PY'
import json
from mlflow import MlflowClient
c = MlflowClient(tracking_uri="http://127.0.0.1:5000")
d = json.load(open("reports/registry.json"))
v = d["versions"]["selected"]["model_version"]
c.set_registered_model_alias("ccpp-power", "champion", v)
actual = c.get_model_version_by_alias("ccpp-power", "champion")
print("champion ->", actual.version)
assert str(actual.version) == str(v)
PY
```

В исходном опыте ожидается champion → 2. В UI у version 2 отображается `@champion`. `register()` назначает alias автоматически; `export()` тоже назначает и проверяет его для выбранной роли.

Состояние alias в ходе демонстрации менялось: после обучения — 2, перед v1 — 1, перед v2 — 2, при откате — 1, после Compose — снова 2. Последнее назначение видно в `reports/logs/compose.log`.

URI `models:/ccpp-power@champion` разрешается через текущее назначение alias. URI `models:/ccpp-power/2` фиксирует версию. Запущенный контейнер читает собственный локальный файл и не переключится автоматически вслед за alias.

П18. В каком порядке выполняются экспорт модели и её итоговая оценка? Какие команды для этого используются?

В нашем фактическом коде итоговая оценка выполняется до Docker-экспорта. Сначала обучаются шесть Pipeline и сохраняются в MLflow; затем по validation выбирается модель, создаются Registry-версии и champion; `evaluate()` загружает фиксированную selected-версию и оценивает test. После этого этапы v1/v2 экспортируют нужные зарегистрированные модели для контейнеров.

```
# После подготовки данных и запуска MLflow, в чистом опыте:
.venv/bin/python src/workflow.py train
.venv/bin/python src/workflow.py register
.venv/bin/python src/workflow.py evaluate
.venv/bin/python src/workflow.py export --role selected --label v2
```

Высокоуровнево первые три операции входят в `bash scripts/run.sh experiments`, экспорт selected — в `run.sh v2`. Оценка использует `models:/ccpp-power/<номер selected>` и не переобучает модель. Результат пишется в `reports/test_metrics.json`, прогнозы — в `reports/test_predictions.csv`, test-метрики — в run победителя.

Технически возможна и схема «выбрать → экспортировать → оценить тот же экспорт». Главное — до test зафиксировать выбор и не изменять веса/преобразования. Нельзя утверждать, что именно так было в нашей реализации: здесь оценивался Registry-артефакт, а перенос в joblib был следующим этапом. Проверка API после экспорта — отдельная функциональная проверка совпадения, а не новый подбор модели по test.

П19. Какие файлы появляются после экспорта? Для чего нужны metadata.json, sample_request.json и expected_predictions.json?

В `exported_models/v1/` либо `exported_models/v2/` создаются четыре файла:

Файл	Назначение
model.joblib	Сериализованный sklearn Pipeline: преобразование и регрессор
metadata.json	Имя/версия Registry, run_id, алгоритм, параметры, SHA-256, признаки, единицы, validation MAE
sample_request.json	Корректный JSON-пакет из первых трёх строк validation
expected_predictions.json	Предсказания этого пакета локальным экспортом, в MW

`metadata.json` позволяет установить происхождение модели и проверить, какая версия реально загружена. При старте API вычисляет SHA-256 joblib и сравнивает с метаданными. `sample_request.json` используется как одинаковый вход для локальной модели и API. `expected_predictions.json` хранит эталон; сравнение выполняется с численным допуском, а не строгим равенством float.

Пример назначения папок: v1 содержит StandardScaler+Ridge, v2 — passthrough+RandomForestRegressor. Это не номера версий Python и не директории MLflow-сервера: они являются снимками для Docker.

В ZIP результатов для передачи и в Git включены три JSON-файла, но не большие joblib. Факт экспорта подтверждают логи и метаданные; бинарные артефакты сохранены

отдельно в остановленном MLflow backup. Наличие JSON само по себе не позволяет загрузить модель: для инференса нужен соответствующий joblib.

П20. Какие инструкции содержит ваш Dockerfile? Что выполняют FROM, WORKDIR, COPY, RUN, USER, EXPOSE и CMD?

Dockerfile собирает среду инференса. Ключевые инструкции:

```
ARG BASE_IMAGE=python:3.12-slim
FROM ${BASE_IMAGE}
ENV MODEL_DIR=/app/model
WORKDIR /app
COPY requirements-runtime.lock.txt .
RUN python -m pip install --no-cache-dir -r requirements-runtime.lock.txt
COPY src/app.py /app/app.py
ARG MODEL_PATH=exported_models/v1
COPY ${MODEL_PATH}/ /app/model/
RUN useradd --create-home appuser && chown -R appuser:appuser /app
USER appuser
EXPOSE 8000
CMD ["python", "-m", "uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Это сокращённая запись: полный файл также задаёт PYTHONUNBUFFERED, PYTHONDONTWRITEBYTECODE, опциональный CA secret для pip и HEALTHCHECK.

- `FROM` выбирает базовый образ; в наших сборках BASE_IMAGE закреплён digest.
- `WORKDIR` задаёт каталог для следующих инструкций и запуска процесса.
- `COPY` переносит выбранные файлы из контекста сборки в образ.
- `RUN` выполняет команду во время сборки, результат входит в слой образа.
- `USER` задаёт пользователя для последующих команд и процесса контейнера; appuser вместо root уменьшает полномочия приложения.
- `EXPOSE` документирует порт контейнера, но не публикует его на хосте.
- `CMD` задаёт команду по умолчанию при запуске контейнера; JSON-форма передаёт аргументы напрямую, без промежуточного shell.

`ARG` передаёт настройку сборщику, `ENV` задаёт переменную среды образа и контейнера. В нашем случае MODEL_PATH — build argument, MODEL_DIR — runtime переменная. HEALTHCHECK задаёт периодическую команду проверки; её точные интервалы и отличие от качества модели разобраны в П29 и Т19.

Установка зависимостей расположена до COPY модели, поэтому при смене модели Docker может повторно использовать кеш слоя пакетов. Образ не содержит запущенного MLflow: API загружает локальный Pipeline.

П21. Как собрать Docker-образ с заданным тегом? Что означает точка в конце команды сборки?

После экспорта v2 команда для нашего Dockerfile:

```
sudo docker build \
  --build-arg BASE_IMAGE="$(cat reports/base-image.txt)" \
  --build-arg MODEL_PATH=exported_models/v2 \
  -t ccpp-api:v2 .
```

`-t` задаёт имя и тег образа. `--build-arg MODEL_PATH` выбирает каталог экспорта, иначе по умолчанию копируется v1. Последняя точка — **контекст сборки**, то есть текущая папка

проекта. Это не точка завершения команды. Из контекста Docker читает Dockerfile и разрешённые `.dockerignore` файлы.

Относительный путь COPY ищется внутри контекста, а не в произвольном каталоге компьютера. Если запустить сборку из другой папки, нужный экспорт/lock-файл может не найтись. Флаг `-f` выбирает Dockerfile, но сам по себе не меняет контекст.

`run.sh v1` сначала получает `python:3.12-slim`, записывает RepoDigest в `reports/base-image.txt`; обе версии строятся с этим digest. Изменяемый тег base сам по себе не гарантировал бы тот же исходный образ спустя время. Высокоуровневая команда `bash scripts/run.sh v2` выполняет и экспорт, и сборку, и замену контейнера, поэтому не равнозначна одной операции `docker build`.

П22. Как проверить загрузку экспортированной модели внутри образа до запуска API?

Можно создать краткоживущий контейнер с Python вместо Uvicorn, загрузить joblib из самого образа и сделать контрольное предсказание. Для ccpp-api:v2:

```
sudo docker run --rm -i --entrypoint python ccpp-api:v2 - <<'PY'
from pathlib import Path
import hashlib
import json
import joblib
import numpy as np
import pandas as pd

root = Path("/app/model")
meta = json.loads((root / "metadata.json").read_text())
actual_sha = hashlib.sha256((root / "model.joblib").read_bytes()).hexdigest()
assert actual_sha == meta["model_sha256"]
model = joblib.load(root / "model.joblib")
body = json.loads((root / "sample_request.json").read_text())
frame = pd.DataFrame(body["instances"], columns=meta["features"])
pred = model.predict(frame)
expected = json.loads((root / "expected_predictions.json").read_text())
np.testing.assert_allclose(pred, expected["predictions_MW"], rtol=1e-10, atol=1e-9)
print("Loaded version:", meta["model_version"])
print("Predictions, MW:", pred.tolist())
PY
```

`--entrypoint python` меняет исполняемую программу, `-` читает код из stdin, `-i` сохраняет stdin, `--rm` удаляет тестовый контейнер после завершения. Порт не публикуется: HTTP-сервер вообще не запускается.

Проверяется именно артефакт внутри образа, а не случайно оставшаяся модель на хосте. В успешном случае завершение с кодом 0 подтверждает доступность файлов, хеш, совместимость библиотек и работоспособность predict. Это полезная дополнительная команда для защиты; не нужно утверждать, что она присутствовала в исходном журнале. В выполненной работе загрузку проверяли startup API и дальнейшее сравнение через HTTP.

П23. Как запустить контейнер в фоновом режиме с публикацией порта только на локальном адресе компьютера?

```
sudo docker run -d \
  --name ccpp-lab17-api --label lab=ccpp-17 \
  -p 127.0.0.1:8000:8000 \
  ccpp-api:v2
```

`-d` запускает контейнер в фоне. `--name` задаёт удобное имя для inspect/logs, label позволяет отличать контейнер лабораторной от чужих. В `-p` первая часть — IP хоста, затем порт хоста, затем порт контейнера. Запрос к 127.0.0.1:8000 на Ubuntu перенаправляется на порт 8000 контейнера.

Внутри образа Uvicorn слушает **0.0.0.0:8000**, чтобы принимать трафик, приходящий через сетевой интерфейс контейнера. Если слушать только 127.0.0.1 внутри контейнера, опубликованный порт обычно не обеспечит нужную доступность. Ограничение внешнего доступа задаётся отдельно на стороне хоста.

Имя и порт должны быть свободны. `run.sh` перед заменой проверяет label и удаляет только свой предыдущий контейнер. `-d` не означает healthy: затем нужны inspect healthcheck, `/health`, `/metadata` и реальный POST /predict.

П24. Как посмотреть работающие и остановленные контейнеры? Как получить логи конкретного контейнера?

```
sudo docker ps
sudo docker ps -a
sudo docker logs --tail 100 ccpp-lab17-api
sudo docker logs -f ccpp-lab17-api
sudo docker inspect ccpp-lab17-api --format '{{json .State}}'
```

`docker ps` показывает работающие контейнеры, `-a` добавляет завершившиеся. `logs` читает stdout/stderr основного процесса, `--tail` ограничивает объём, `-f` следит за новыми сообщениями. Ctrl+C завершает просмотр logs, а не сам контейнер. В inspect можно посмотреть статус, exit code, ошибку и health.

Если приложение сразу остановилось, контейнер может отсутствовать в обычном ps, но будет в ps -a. Тогда смотрю startup traceback: например, отсутствует joblib, не совпал хеш или версии библиотек. Имя можно заменить container ID.

`docker logs` не читает автоматически все файлы внутри контейнера и не даёт логи уже удалённого контейнера. После перехода на Compose используется `docker compose -p ccpp-lab17 logs api`, поскольку старое ручное имя удалено. Файлы `reports/logs/v1.log` и подобных этапов — журналы оркестратора/проверок; это не полная копия stdout/stderr Uvicorn из удалённого контейнера.

П25. Как проверить /health и /metadata через curl? Какие сведения должны содержать ответы?

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/metadata \
  | .venv/bin/python -m json.tool
```

После финального запуска Compose `/health` возвращал смысловой результат:

```
{"status": "ok", "model_version": "2"}
```

Ответ возможен только после успешного lifespan: файл модели уже проверен по SHA-256 и загружен. Это полезный readiness-подобный сигнал, но не расчёт качества модели и не проверка каждого возможного входа.

`/metadata` возвращает имя `ccpp-power`, номер версии, run_id, алгоритм, гиперпараметры, model_sha256, порядок признаков, цель PE, единицы и validation MAE. Для v2 ожидается RF и run_id `503b388935b047bca0fa775e9603e454`. Сверяю ответ с `exported_models/v2/metadata.json`, а не только с надписью «сервис работает». Так можно заметить ошибочно запущенный v1.

`curl -f` возвращает ошибку при HTTP 4xx/5xx. Для изучения тела специально некорректного запроса удобнее вариант без `-f`, например `curl -i`, чтобы увидеть и статус, и JSON объяснения.

П26. Как отправить POST-запрос с JSON из файла через curl? Как выполнить аналогичный запрос через /docs?

Для запущенного v2 использую сформированный экспортом файл:

```
curl -sS -i -X POST http://127.0.0.1:8000/predict \
  -H 'Content-Type: application/json' \
  --data-binary @exported_models/v2/sample_request.json
```

`-X POST` задаёт метод, `-H` — тип содержимого, знак `@` означает чтение тела из файла, `-i` добавляет заголовки со статусом. В sample_request три validation- наблюдения; сервер возвращает три числа в `predictions_MW` и model_version.

В браузере открываю `http://127.0.0.1:8000/docs`, раскрываю POST /predict, нажимаю Try it out, вставляю JSON и нажимаю Execute. Для одиночного примера:

```
{"instances": [{"AT": 20.0, "V": 50.0, "AP": 1010.0, "RH": 70.0}]}
```

В нашем ручном запросе получено HTTP 200 и:

```
{"model_version": "2", "predictions_MW": [454.9589200000001], "unit": "MW"}
```

Скриншот `13-api-correct.png` показывает и сформированный curl, и ответ. Этот вход отличается от sample_request из трёх строк, поэтому сравнивать его число с первым числом другого пакета как проверку совпадения нельзя. Адрес /docs относится к FastAPI на 8000, а не к MLflow на 5000.

П27. Как проверить обработку некорректных запросов и совпадение предсказаний API с локальной моделью?

Готовая функциональная проверка:

```
.venv/bin/python src/workflow.py check --label v2 --phase v2
```

Она ждёт `/health`, сверяет `/metadata` с конкретным экспортом, загружает локальный joblib, делает predict для того же sample_request, отправляет его в API и сравнивает числа через:

```
np.testing.assert_allclose(
    actual["predictions_MW"], expected,
    rtol=1e-10, atol=1e-9
)
```

Проверяются 8 классов ошибок: пустой пакет, отсутствующий признак, число строкой, лишнее поле PE, RH=101 вне согласованного диапазона, неверный верхний ключ data, 1001 строка вместо разрешённых 1000, нечисловой NaN. Везде ожидается 422. Для ручной демонстрации удаляю RH и проверяю `Field required`, как на `14-api-invalid.png`. Сами JSON и статусы сохранены в `reports/api_<этап>.json`.

Не проверяю ответы только оператором `==`: у многопоточного RF сумма деревьев может отличаться в последних битах. В наших v2 и Compose разница составила один ULP — $5{,}68434\times10^{-14}$ МВт. Именно точное сравнение вызвало ложный `update_verified=false` старого сборщика. Исходный флаг сохранён; отдельная проверка с исходным допуском, одинаковыми image ID/хешами модели подтвердила совпадение. Допуск нельзя расширять произвольно, чтобы скрыть существенное расхождение или другую модель.

П28. В каком порядке выполняются обновление модели, экспорт, сборка нового образа и замена контейнера? Как затем выполнить откат?

Сначала новая модель уже должна быть обучена, проверена по принятому протоколу и зарегистрирована. В нашей демонстрации обе версии обучены заранее. Обновление выполняет последовательность:

1. Выбрать Registry-версию 2 и назначить champion.
2. Экспортировать её Pipeline и метаданные в отдельный каталог v2.
3. Собрать `ccpp-api:v2` с этим экспортом.
4. Заменить ручной контейнер v1 контейнером v2 на том же порту.
5. Проверить версию, корректные/ошибочные запросы, совпадение и healthy.

```
sudo -v
bash scripts/run.sh v2
bash scripts/run.sh rollback
```

`rollback` возвращает champion на version 1 и запускает **старый готовый образ ccpp-api:v1**. Он не вызывает fit, export или docker build. Поэтому важно заранее сохранить предыдущий образ и соответствующий локальный эталон. Возврат одного alias без замены контейнера не меняет уже загруженную модель.

В доказательствах image ID v1 и отката совпадают; отличаются только container ID новых экземпляров. Первый прогноз одного пакета: 463,955885 МВт в v1, 465,653520 МВт в v2 и снова 463,955885 МВт после отката. Для реальной системы нужны также совместимость схем, проверка до переключения трафика и план отката конфигурации; здесь показана учебная замена с возможным коротким перерывом, а не гарантированное обновление без простоя.

П29. Как запустить сервис через Docker Compose и проверить статус healthcheck? Как посмотреть его логи после удаления ручного контейнера?

Штатная команда проекта:

```
sudo -v
bash scripts/run.sh compose
```

Она возвращает champion на selected, удаляет свой ручной контейнер, затем выполняет `compose up --wait`. Для ручной демонстрации после освобождения порта команды выглядят так:

```
sudo docker compose -p ccpp-lab17 up -d --wait --wait-timeout 120
sudo docker compose -p ccpp-lab17 ps
sudo docker compose -p ccpp-lab17 logs --tail 100 api
sudo docker compose -p ccpp-lab17 logs -f api
```

Сервис в compose.yaml называется **api**, проект — **ccpp-lab17**. После удаления `ccpp-lab17-api` его имя не используется для новых логов: Compose создал собственный контейнер `ccpp-lab17-api-1`. Надёжнее обращаться к сервису через Compose, а не предполагать имя контейнера.

Подробная проверка health:

```
container_id=$(sudo docker compose -p ccpp-lab17 ps -q api)
sudo docker inspect "$container_id" \
  --format '{{json .State.Health}}'
```

Ожидается Status=healthy. В конфигурации interval=5s, timeout=3s, start_period=20s, retries=12; команда проверяет GET /health. Затем отдельно проверяются предсказания. `restart: unless-stopped` сам по себе не перезапускает контейнер только из-за статуса unhealthy. Для завершения учебного запуска: `sudo docker compose -p ccpp-lab17 down`. Логи уже удалённого ручного контейнера можно посмотреть лишь если их сохранили заранее; Compose logs показывает новый сервис, а не восстанавливает старые логи.

II. Вопросы по теории для защиты лабораторной работы

T1. Что такое MLOps? Какие этапы жизненного цикла модели он охватывает?

MLOps — организация разработки и эксплуатации ML-систем, которая объединяет работу с данными и моделями с практиками программной инженерии и эксплуатации. Цель — регулярно и контролируемо доставлять проверенную модель в работающий сервис, понимать её происхождение, замечать деградацию и уметь восстановиться. Это не название одной библиотеки и не только упаковка pickle в Docker.

Жизненный цикл включает постановку задачи и критерия успеха; получение и версионирование данных; проверку качества данных и построение признаков; обучение и сравнение экспериментов; независимую оценку; регистрацию кандидата; сборку и тестирование сервиса; развёртывание; мониторинг, обратную связь, переобучение и вывод устаревшей версии из эксплуатации.

Автоматизация часто строится вокруг CI — проверок кода/контрактов при изменениях, CD — доставки проверенного артефакта, CT — контролируемого переобучения при новых данных или деградации. Каждое переобучение не должно автоматически превращаться в production без оценки и согласованных критериев допуска.

В нашей работе выполнен учебный фрагмент цикла: зафиксированы CCPP и split, сравнены 6 run, выбрана и зарегистрирована модель, создан API и Docker, проверены обновление, откат и Compose. Полноценные CI/CD, постоянный мониторинг дрейфа и автоматическое переобучение не реализованы; их нельзя приписывать проекту только из-за присутствия MLflow и Docker.

T2. Чем разработка модели машинного обучения отличается от её эксплуатации?

При разработке основной вопрос — как получить модель, которая решает задачу с нужным качеством на корректно сформированных проверочных данных. Исследуются данные, признаки, гиперпараметры, ошибки и ограничения оценки.

При эксплуатации модель становится частью программной системы. Нужно принимать реальные запросы, поддерживать согласованный формат, обеспечивать доступность и приемлемую задержку, управлять ресурсами и версиями, отслеживать изменения данных и качество при появлении фактических ответов. Ошибка развертывания может повредить систему даже при отличной offline-метрике.

Разработка	Эксплуатация
Поиск алгоритма и гиперпараметров	Загрузка конкретного проверенного артефакта
Разовая или пакетная оценка	Многократные запросы и ограничения времени ответа
Исследование данных	Контроль входных контрактов и новых распределений
Сравнение кандидатов	Обновления, наблюдаемость и откат

В нашей работе 6 MLflow run относятся к разработке. FastAPI, валидация, healthcheck, image IDs и откат — к доставке и учебной эксплуатации. После развёртывания сервис не обучается на каждом запросе, а только вызывает predict у загруженного Pipeline. Хороший R^2 не отменяет требований к API, и HTTP 200 не доказывает, что прогноз точен для новых режимов электростанции.

T3. Что означает воспроизводимость эксперимента? Какие сведения необходимы для повторения результата?

Воспроизводимость означает возможность повторить методику и получить согласованный результат, опираясь на сохранённые данные, код, настройки и среду. Нужно различать восстановление исходного готового артефакта и повторное обучение: первое возвращает тот же файл, второе может дать небольшие численные отличия.

Для повторения нужны исходная версия данных и её источник, контрольные суммы, правила очистки, порядок/индексы train-validation-test, список признаков и цель, единицы, код и его версия, гиперпараметры, seeds, версии Python/библиотек, важные особенности платформы, протокол вычисления метрик и критерий выбора. Для сервиса также нужны преобразования, веса, конфигурация API и точный образ.

В нашем проекте сохранены ZIP/XLSX, SHA-256, split с row_id, seed=42, lock-файлы, код, JSON метрик и Registry, прогнозы и image IDs. Метрики test можно пересчитать без повторного fit по `test_predictions.csv`. Но SQLite Registry и joblib вынесены в отдельный backup: для восстановления именно старых run они нужны.

Один seed не фиксирует версии библиотек, данные и порядок операций. Потоки, разные процессоры и реализации линейной алгебры могут менять последние биты. Например, одинаковый RF дал в v2 и Compose отличие $5{,}68\times10^{-14}$ МВт. Поэтому заранее задаётся численный допуск; он не означает разрешение произвольных отличий. Порог метрик и точность сравнения выбираются осмысленно до проверки.

T4. Что такое MLflow? Для чего предназначены Tracking, Models и Model Registry?

MLflow — платформа инструментов для управления экспериментами и жизненным циклом моделей. В нашей работе использована версия 3.5.0.

Компонент	Что делает	Пример нашего проекта
Tracking	Хранит experiment/run, параметры, метрики, теги, ссылки и файлы	6 run с validation MAE
Models	Стандартизирует сохранение модели, её flavours, схему и окружение	sklearn Pipeline со signature и input_example
Model Registry	Даёт модели имя, версии, метаданные и aliases	ccpp-power, версии 1/2, champion

MLflow Model обычно содержит описание `MLmodel`, файлы модели и зависимости; flavour сообщает способ загрузки/предсказания, например sklearn или pyfunc. В MLflow 3 логированная модель имеет собственный model ID, поэтому она не тождественна run. Один run может создавать несколько моделей.

В нашей конфигурации SQLite хранит метаданные, файловый artifacts-каталог — содержимое артефактов. Tracking server принимает запросы клиентов и проксирует артефакты. Это удобная учебная схема; для большого многопользовательского контура обычно выбирают более подходящие БД, хранилище и контроль доступа.

MLflow не заменяет сам алгоритм обучения, FastAPI или Docker. В проекте обучает sklearn, HTTP-контракт реализует FastAPI, а контейнер доставляет экспортированный снимок. После сборки предсказания не зависят от живого MLflow.

T5. Чем experiment, run, сохранённая модель и Registry-версия отличаются друг от друга?

Experiment — логическая группа связанных запусков. **Run** — один сеанс обучения/измерения с собственным ID и результатами. **Сохранённая модель** — конкретный вычислительный артефакт, способный выдавать предсказания. **Registry-версия** — запись, которая связывает такой артефакт с именованной моделью и её номером в реестре.

В нашей работе:

- experiment: `lab4-ccpp-variant17`;
- run: `rf-unlimited`, ID `503b388935b047bca0fa775e9603e454`;
- logged model: `models:/m-7a02a502abe74b62b1bd4704453e6475`;
- Registry: `ccpp-power`, version 2;
- экспорт: `exported_models/v2/model.joblib`;
- образ сервиса: `ccpp-api:v2`.

Номер 2 в Registry и тег v2 Docker согласованы нами для демонстрации, но технически не вычисляются друг из друга. В следующем опыте новый Registry может присвоить номер 5, а имя образа выбирается отдельно.

Run не обязательно заканчивается регистрацией; неудачные/исследовательские запуски тоже полезно сохранять. Регистрация не переобучает модель, а создаёт запись о существующем артефакте. Экспорт переносит его в формат/папку доставки. Изменение метаданных run не означает появление новых обученных весов.

Т6. Чем параметры, гиперпараметры, метрики, теги и артефакты отличаются друг от друга?

Сущность	Смысл	Пример
Обучаемые параметры модели	Получаются из данных во время fit	Коэффициенты Ridge, пороги и значения листьев RF
Гиперпараметры	Задают способ обучения до fit	alpha, n_estimators, max_depth, min_samples_leaf
MLflow params	Зафиксированные настройки запуска	Гиперпараметры, seed, размер train, preprocessing
Метрики	Численные измеренные результаты	val_mae, val_rmse, val_r2, fit_seconds
Теги	Метаданные для поиска и объяснения	variant=17, target_unit=MW, dataset_sha256
Артефакты	Файлы и структурированные результаты	Pipeline, график, split.json, исходник, lock-файл

Термин «параметр» зависит от контекста: MLflow param `alpha` является гиперпараметром sklearn, а не обученным коэффициентом. Обученные веса обычно входят в файл модели, а не записываются все отдельными log_param.

Метрика без указания выборки неоднозначна. Поэтому используются имена `val_mae` и `test_mae`; у MAE/RMSE единицы MW, у R^2 единиц нет. Настройка `n_jobs` в первую очередь управляет вычислительным параллелизмом, а не сложностью модели, но её тоже важно фиксировать как настройку запуска.

Теги не выбирают победителя сами: `selection_metric=val_mae` документирует правило, а код сравнивает значения. Артефакты нужны, чтобы можно было перейти от записи «MAE хорошая» к конкретным файлам и воспроизводимому эксперименту.

Т7. Что такое происхождение модели — model lineage? Зачем связывать модель с кодом, данными и экспериментом?

Model lineage — цепочка происхождения: какие исходные данные и их версия, правила подготовки, split, код, окружение и настройки привели к конкретной модели, а затем к её развёртыванию. Она позволяет ответить не только «что сейчас работает», но и «почему получилось именно это».

Для нашего победителя цепочка выглядит так: исходный CCPP ZIP → Sheet1 → дедупликация и split=42 → run RF → logged model → Registry version 2 → joblib с SHA-256 → образ v2 с image ID → конкретный контейнер/API. `metadata.json` связывает run_id, Registry-версию, алгоритм и хеш файла; контейнерные JSON связывают этап с образом. Артефакты run содержат код, зависимости и split.

Lineage помогает найти ошибку данных, сравнить кандидатов, доказать отсутствие подмены, отследить затронутые развёртывания и выполнить управляемый откат. Если известен только тег `latest`, восстановить цепочку часто невозможно.

В нашем run сохранён исходный Python-файл и хеши данных/split. Наличие отдельного git_commit tag в каждом историческом run не заявляется: это полезное улучшение будущего процесса. Важно хранить не только хеш, но и доступную копию объекта, поскольку SHA-256 не восстанавливает потерянные данные.

T8. Для чего нужны версии моделей и aliases? Чем изменяемая ссылка отличается от конкретного номера версии?

Версии отделяют кандидатов и позволяют явно выбрать, сравнить, воспроизвести или вернуть конкретный артефакт. Alias — удобное именованное указание роли: например, champion означает выбранную основную модель. Это соглашение команды, а не встроенная гарантия, что MLflow автоматически выбрал лучший алгоритм.

```
models:/ccpp-power/2          — конкретная Registry-версия
models:/ccpp-power@champion   — текущее назначение alias
```

При переназначении champion новая загрузка по alias может получить другую версию. Ссылка на номер 2 продолжает указывать на запись версии 2. Для строгой доставки фиксируют также хеши/адреса файлов и образа, поскольку надёжность версии опирается на сохранность её артефактов.

У нас alias сначала назначался RF версии 2, временно переходил на 1 для v1 и отката, затем возвращался на 2 при Compose. Запущенный контейнер не следит за Registry: он загрузил локальный joblib. Чтобы реально обновить сервис, нужны экспорт, новый образ и замена контейнера.

Aliases удобны для рабочих процессов, а конкретная версия и digest — для аудита и воспроизводимости. Перед переносом alias нужно проверить кандидата; права изменить ссылку не являются доказательством качества новой модели.

T9. Зачем разделять данные на train, validation и test? Какую роль выполняет каждая выборка?

Часть	Роль	Что разрешено
Train	Оценить параметры модели и преобразований	fit Pipeline, настройка scaler на этих данных
Validation	Сравнить кандидатов и выбрать гиперпараметры	Многократное измерение для выбора конфигурации
Test	Независимо оценить уже принятое решение	Итоговая проверка после фиксации выбора

Без отдельного validation настройка по test постепенно подгоняла бы решение к тестовой выборке и делала оценку оптимистичной. Validation тоже может переобучаться при большом числе попыток, поэтому финальный test отделяют.

В нашем проекте 5715 train, 1906 validation, 1906 test. StandardScaler+Ridge и RF обучаются на train, шесть вариантов ранжируются по validation MAE. Затем выбранный RF оценивается на test без переобучения. Это проще связать с точным артефактом Registry. Иная допустимая методика — после выбора обучить на train+validation и оценить на нетронутом test; тогда нужно зарегистрировать и описать уже этот новый артефакт. Мы так не делали.

Для временных, групповых и зависимых наблюдений случайное построчное деление может быть неверным. У нас одинаковые X группируются, а временных меток нет. Поэтому оценка относится к прогнозу по доступным параметрам в данном наборе, а не к доказанному прогнозированию будущего периода работы электростанции.

T10. Что такое утечка данных? Как она возникает при подготовке признаков и выборе модели?

Утечка — использование информации, которая не должна быть доступна модели или процессу выбора в момент предсказания/оценки. Из-за неё проверочная ошибка оказывается лучше, чем на действительно новых данных.

Типичные случаи: включить цель или её следствие в X; вычислить scaler/imputer по всему набору; выбирать признаки по test; многократно менять гиперпараметры после просмотра test; допустить дубликаты одного наблюдения в train и test; для временных данных использовать будущую информацию или случайно перемешать соседние зависимые измерения без обоснования.

Для CCPP особенно опасно объединить пять перемешанных листов: одинаковые исходные точки окажутся в разных частях. Ещё одна ошибка — включить PE или служебный row_id в признаки. Наш код использует только AT,V,AP,RH, удаляет полные дубликаты, группирует одинаковые X и помещает StandardScaler в Pipeline. Test не используется для подбора шести конфигураций.

Изучение схемы и документированных единиц источника отличается от обучения параметров преобразования по test. Например, факт RH=100.16% из источника нужен для корректного контракта измерений; среднее и стандартное отклонение для Ridge всё равно оцениваются только на train. Проверка утечки должна учитывать не только код fit, но и весь процесс принятия решений.

T11. Что такое Pipeline? Почему преобразования данных необходимо сохранять вместе с моделью?

`sklearn.pipeline.Pipeline` объединяет последовательность преобразований и финальный estimator в один объект с методами fit/predict. При fit каждый обучаемый шаг получает данные соответствующей обучающей части; при predict применяются уже зафиксированные преобразования, а затем регрессор.

Наши варианты:

```
Pipeline([
    ("preprocess", StandardScaler()),
    ("regressor", Ridge(alpha=0.1))
])
```

Для RF вместо scaler задан `passthrough`. Масштабирование меняет смысл штрафа Ridge, поэтому необходимо сохранять средние и масштабы, полученные на train. Если на сервере заново обучить scaler на запросе или забыть стандартизацию, модель будет получать другой вход и предсказания изменятся.

Сохранение Pipeline делает поставку единой: веса и преобразования не нужно согласовывать вручную по разным файлам. Это также помогает правильно применять CV: scaler заново обучается внутри обучающего фолда, а не на всех наблюдениях.

Однако Pipeline сам по себе не исправит плохой split или подготовленную заранее с утечкой таблицу. Нужно также фиксировать имена, порядок, типы и единицы признаков. FastAPI создаёт DataFrame в порядке AT,V,AP,RH, соответствующем fit.

T12. Что такое сериализация модели? Какие проблемы совместимости и доверия возникают при её загрузке?

Сериализация переводит объект с его состоянием в сохраняемый формат, а десериализация восстанавливает объект для использования. В проекте MLflow сохраняет sklearn-модель, а экспорт дополнительно использует `joblib.dump(model, path)` и `joblib.load(path)` для Pipeline.

Нужно сохранять не только класс алгоритма, но и обученные коэффициенты/деревья, состояние preprocessing, схему входа и зависимости. Совместимость зависит от версий Python, sklearn, numpy, joblib, доступности импортируемых классов и иногда бинарной платформы. Не следует считать sklearn pickle универсальным стандартом, гарантированно читаемым любой будущей версией sklearn.

Вопрос доверия: joblib/pickle могут выполнять код при загрузке. Поэтому десериализуют артефакты из доверенного процесса, а не любой присланный файл. Проверка SHA-256 подтверждает совпадение с ожидаемыми байтами, но не делает файл безопасным, если злоумышленник мог заменить и модель, и ожидаемый хеш. Нужны доверенное хранилище, контроль доступа и при необходимости подписи.

У нас runtime lock согласован с обучением, модель поставляется в образе, startup проверяет хеш, затем делает joblib.load. Проверка predict внутри образа и API-сравнение дополнительно выявляют проблемы совместимости. Разные байтовые файлы могут задавать близкие вычисления; равенство хеша и численная близость предсказаний отвечают на разные вопросы.

T13. Что такое Docker? Чем образ, контейнер и Dockerfile отличаются друг от друга?

Docker — набор средств сборки, хранения и запуска контейнеризированных приложений. Контейнер использует изоляцию процессов/ресурсов и файловую систему образа, а также ядро Linux-хоста.

Dockerfile — текстовый рецепт сборки. **Образ** — упакованный результат: слои файлов, конфигурация запуска, метаданные. **Контейнер** — экземпляр образа с собственным процессом, состоянием и записываемым слоем. Один образ можно запустить много раз; остановка контейнера не удаляет автоматически образ.

В нашей работе Dockerfile устанавливает runtime-зависимости, копирует API и конкретный Pipeline. `ccpp-api:v1` и `ccpp-api:v2` — разные снимки сервиса. При откате создан новый контейнер из прежнего v1: поэтому container ID новый, а image ID прежний. Исходный Dockerfile и тег сами по себе не доказывают, что в контейнере оказался нужный файл: это проверяют хеш и metadata/API.

Контейнеры не являются полной копией отдельной машины с независимым ядром. В данном случае Docker работает внутри Ubuntu-виртуалки на Mac M1, контейнеры используют Linux-ядро этой Ubuntu. User-space базового Python-образа может отличаться от Ubuntu, что не отменяет факта выполнения работы на Ubuntu-хосте.

T14. Чем контейнеризация отличается от виртуальной машины и виртуального окружения Python?

Средство	Что изолирует	Что остаётся общим
Python venv	Набор Python-пакетов и выбор интерпретатора	ОС, ядро, пользователь, сеть, системные библиотеки
Контейнер	Процесс, файловую среду приложения, часть ресурсов/сети	Ядро хоста и физическое оборудование
Виртуальная машина	Гостевую ОС с собственным ядром	Физический хост и гипервизор

Venv достаточно для независимых пакетов двух Python-проектов, но он не поставляет системную ОС и не является защитной песочницей. Контейнер удобен, когда нужно переносить Python вместе с системными библиотеками и командой запуска. Виртуалка нужна, если требуется отдельная ОС/ядро, например Linux на компьютере с macOS.

У нас используются все три уровня: Mac M1 → Ubuntu 22.04 ARM64 в виртуальной машине → .venv для обучения и Docker-контейнер для инференса. Их задачи не дублируют друг друга. Python .venv не заменяет Docker, а Docker-образ не заменяет ОС, которая обеспечивает его выполнение.

Архитектура также важна: ARM64-библиотеки и образы должны соответствовать ARM64. Мультиархитектурный тег выбирает вариант платформы; принудительный AMD64 обычно потребовал бы эмуляции. Контейнеризация не делает код независимым от архитектуры и не гарантирует побитовое равенство вычислений на разных CPU.

T15. Что такое контекст сборки, слои образа и кеш Docker? Для чего нужен .dockerignore?

Контекст — набор файлов, доступных сборщику. В `docker build ... .` это текущая папка с применением правил `.dockerignore`. COPY берёт файлы из этого набора. Выбор Dockerfile через `-f` и выбор контекста — разные действия.

Слои описывают изменения файловой системы между шагами сборки. Docker повторно использует подходящие результаты из кеша, если инструкция и её входы не изменились. После изменения раннего шага следующие зависящие от него шаги могут потребовать пересборки. Кеш ускоряет работу, но не заменяет фиксацию источников и зависимостей.

В нашем Dockerfile сначала копируется runtime lock и ставятся пакеты, затем копируются app и экспорт. Поэтому новая модель не требует заново устанавливать те же библиотеки. Разные модельные слои образов могут ссылаться на общие слои базовой среды.

`.dockerignore` уменьшает контекст, ускоряет передачу, исключает случайное попадание .venv, данных, MLflow и лишних файлов. В проекте используется правило «исключить всё, разрешить runtime lock, app.py и экспортированные модели»; COPY выбирает конкретную папку v1 или v2. Файл не заменяет .gitignore: первый относится к Docker, второй — к отслеживанию Git.

Если секрет уже скопирован в слой, удаление его следующей RUN не гарантирует исчезновения из истории образа. Для временного доверенного CA в сборке предусмотрен BuildKit secret, который не копируется в постоянные слои.

T16. Чем тег образа отличается от Image ID и digest? Как это связано с воспроизводимостью развёртывания?

Тег — читаемое имя вроде `ccpp-api:v2` или `python:3.12-slim`. Его можно переназначить: через месяц тот же тег может обозначать другое содержимое. **Image ID** — идентификатор конкретного образа, который сообщает Docker. **Registry digest** — адресуемый по содержимому хеш манифеста или индекса реестра, например `repository@sha256:....`

В классическом Docker Image ID обычно связан с хешем конфигурации образа, а RepoDigest — с манифестом. У современных image store и мультиархитектурных образов есть отдельные хеши конфигурации, платформенных манифестов и индекса; не нужно считать любой выведенный sha256 одним и тем же видом объекта. В отчёте записаны именно значения `.Id`/`.Image`, возвращённые нашим Docker, и отдельно зафиксирован RepoDigest базового образа.

Идентификатор	Для чего удобен
Тег	Удобный выбор релиза человеком
Image ID	Проверка, что локально использован тот же образ
RepoDigest	Закрепление содержимого при получении из реестра

У v1/отката совпал image ID `sha256:5ad83f412ed49351b1c4fc9e0c0e137041782b93e925b4b6b2bc09fdaa95bbbd`. У v2/Compose — другой общий image ID `sha256:e58f35cdca769d8147a61f5c9fcd7fe1ba4d7ca6ad7aa67c07f37681fecebb9b`. Одинаковый тег без проверки ID не доказал бы возврат старого содержимого. Фиксация образа не фиксирует внешние volumes, переменные и входные данные: для воспроизводимого развёртывания важна и конфигурация запуска.

T17. Как устроена публикация портов контейнера? Чем адреса 127.0.0.1 и 0.0.0.0 отличаются при настройке сервиса?

Процесс сначала должен слушать порт **внутри** контейнера. Затем Docker настраивает перенаправление с адреса/порта хоста на адрес/порт контейнера. В нашей команде `-p 127.0.0.1:8000:8000` первый 8000 относится к Ubuntu, второй — к контейнеру. Они могут различаться, например 8080:8000.

127.0.0.1 — loopback конкретного сетевого пространства. У хоста и контейнера свои loopback. **0.0.0.0** при bind означает все доступные IPv4-интерфейсы данного пространства, а не отдельный удалённый адрес сервера.

Uvicorn внутри контейнера слушает 0.0.0.0:8000, чтобы принимать трафик от Docker-сети. Публикация на 127.0.0.1 хоста ограничивает вход локальным компьютером. Если указать только `-p 8000:8000`, Docker обычно публикует порт на всех интерфейсах хоста; это меняет доступность сервиса снаружи. `EXPOSE 8000` сам по себе не создаёт такое правило.

В нашей виртуализации localhost в Firefox на Mac — это Mac, а localhost в Firefox Ubuntu — Ubuntu. Для обращения с Mac понадобился бы отдельно настроенный доступ к VM/перенаправление или туннель. В работе браузер и curl были внутри Ubuntu, поэтому использовался точный адрес 127.0.0.1.

Т18. Что такое HTTP API модели? Для чего нужны валидация входных данных и описание формата запросов и ответов?

HTTP API — программный интерфейс, через который клиент передаёт данные модели и получает результат по HTTP. Он отделяет способ обучения от клиентов: им достаточно знать адрес, метод и контракт, а не устанавливать sklearn и загружать Python-объект самостоятельно.

В нашем сервисе POST /predict принимает `instances` с AT,V,AP,RH и возвращает `predictions_MW`, `model_version`, `unit`. Pydantic проверяет структуру, обязательные поля, типы, допустимость чисел, V/AP>0, 0≤RH≤100.16 и размер пакета. `extra="forbid"` запрещает лишние поля, `strict=True` не превращает строку "20" в число незаметно. Массив прогнозов сохраняет порядок входных строк.

Документация OpenAPI/Swagger показывает схему и позволяет выполнить пример через /docs. Контракт должен включать не только JSON-типы, но и смысл: например, V измеряется в **сантиметрах**, а не миллиметрах ртутного столба. Значение в неправильных единицах может пройти простую числовую проверку и испортить прогноз, поэтому schema validation не заменяет контроль данных.

HTTP 200 означает успешную обработку запроса, 422 — отклонение по схеме в нашей FastAPI-реализации, 404 — неизвестный путь, 405 — неверный метод. Внутренняя ошибка не должна трактоваться как ожидаемая валидация. Номер модели в ответе помогает связать предсказание с версией и исследовать проблему.

Т19. Что проверяют healthcheck, readiness и liveness? Почему работоспособность сервиса не подтверждает качество модели?

Liveness отвечает, жив ли процесс и способен ли он продолжать работу. **Readiness** — готов ли экземпляр принимать рабочий трафик, например загружена ли модель и доступны ли необходимые зависимости. **Healthcheck** — конкретная периодическая проверка, смысл которой задаёт её реализация.

В оркестраторе liveness может управлять перезапуском, readiness — включением экземпляра в маршрутизацию. В Docker HEALTHCHECK выставляет состояния starting/healthy/unhealthy. Он сам по себе не равнозначен Kubernetes readiness или liveness и не гарантирует автоматическое устранение проблемы. `restart: unless-stopped` реагирует на завершение процесса, а не просто на переход в unhealthy; `compose up --wait` ждёт начальной готовности.

В нашем /health доступен после успешной проверки хеша и загрузки Pipeline. Он возвращает status и версию; запрос не вычисляет MAE и не заново читает все веса при каждом вызове. Для рабочего запроса отдельно выполняется predict и сверяется результат. MLflow не является runtime-зависимостью этого API, поэтому его остановка не должна ломать инференс.

Даже здоровый сервис может содержать слабую/неподходящую модель, принимать данные в неправильных единицах или работать на новом распределении. Качество требует сопоставления прогнозов с правильными истинными PE на релевантных данных. Healthy, низкая задержка и хорошая MAE — разные измерения.

Т20. Что такое мониторинг модели после развёртывания? Зачем отслеживать ошибки, задержки, качество предсказаний и дрейф данных?

Мониторинг — регулярный сбор и анализ сигналов о работе сервиса и модели после выпуска. Его цель — обнаружить техническую неисправность или потерю предметного качества достаточно рано, чтобы проверить причину и принять меры.

Технические сигналы: доступность, частота 4xx/5xx, доля отклонённых запросов, таймауты, частота запросов, использование памяти/CPU, задержки p50/p95/p99. Средняя задержка одна не показывает редкие очень медленные ответы. Для расследования полезны время, request ID, версия модели и статус обработки.

Сигналы данных: пропуски, диапазоны и единицы, распределения AT/V/AP/RH, доли необычных комбинаций. Data drift — изменение распределения входов P(X). Concept drift — изменение связи P(Y|X): одинаковые параметры могут со временем давать другую мощность, например после изменения состояния оборудования. Изменение распределения само по себе ещё не доказывает падение качества.

Сигналы качества: когда приходят фактические PE, считаются скользящие MAE/RMSE и ошибки по группам/режимам; анализируется систематическое смещение. Без истинных ответов нельзя честно измерить текущую MAE только по входам. Для CCPP особенно важны сезонность, режим нагрузки и изменение состояния турбин: исходный набор описывает полную нагрузку 2006–2011 годов.

Порог оповещения должен вести к проверке данных, причин, при необходимости откату или контролируемому переобучению с новой оценкой. В нашем проекте есть разовые smoke-tests, логи и offline-метрики, но нет постоянной промышленной системы мониторинга дрейфа. Это следующий этап развития, а не уже выполненная возможность MLflow Tracking по умолчанию.

III. Краткий план ответа о собственной работе

1. Задача — регрессия PE в МВт по четырём параметрам среды, не классификация.
2. Взял один Sheet1 CCPP; 41 дубль удалил, повторные листы не объединял.
3. Зафиксировал train/validation/test=5715/1906/1906 и seed=42.
4. Сравнил 3 Ridge и 3 RF по validation MAE, scaler находится в Pipeline.
5. Выбрал RF с 250 деревьями; test MAE=2,490805 МВт, test не участвовал в выборе.
6. Зарегистрировал Ridge/1 и RF/2, назначал champion, экспортировал Pipeline.
7. Проверил API с корректным пакетом и 8 ошибочными сценариями; показал 200/422.
8. Собрал два ARM64-образа, сверил API с локальным экспортом.
9. Показал v1→v2→откат v1 с прежним image ID без повторного обучения.
10. Запустил Compose с healthy, сохранил данные, зависимости, логи и скриншоты.

Не называю R^2 «процентом правильных ответов», HTTP 200 — доказательством точности, alias — автоматическим обновлением контейнера, а JSON Registry — резервной копией обученной модели. В отчёте отдельно объяснён ложный флаг сборщика из-за сравнения float через ==; исходные свидетельства не изменены.

IV. Источники и навигация по проекту

- Полный отчёт, выполненный notebook.
- workflow.py: данные, обучение, Registry, экспорт, проверки.
- app.py: schema, startup, endpoints.
- run.sh: этапы и Docker/Compose, MLflow server helper.
- Dockerfile, compose.yaml.
- Исходные результаты и проверки.
- UCI CCPP, DOI 10.24432/C5002N.
- MLflow Tracking, Model Registry.
- sklearn Pipeline, сохранение моделей.
- Dockerfile reference, Docker Compose services.
- FastAPI, Pydantic.

Вопросы и исходная методичка «Лабораторная работа 4. MLOps» предоставлены пользователем. Примеры дополнительных проверок в П11 и П22 обозначены как учебные команды; их выводы не выдаются за новые измерения исходного отчёта.